

ამ დანართში მოცემულია სისტემის საწყისი კოდის ყველაზე მნიშვნელოვანი ფრაგმენტები – სტატუსების უსაფრთხო მართვა (`thread-safe getState/setState`), სენსორისა და Telegram-ის ნაკადები (`tasks`) და ცეცხლის ჩაქრობის ამოცნობის ლოგიკა – მოკლე განმარტებებით. სრული კოდი ხელმისაწვდომია GitHub-ზე.

სისტემის ძირითადი პარამეტრები

ქვემოთ მოცემულია ის ძირითადი პარამეტრები, რომელთა საფუძველზეც მუშაობს სისტემა: ცეცხლის ანთება/ჩაქრობის ტემპერატურული ზღვრები, ცეცხლის ჩაქრობის დასადასტურებლად საჭირო კადრების (`frame-ების`) რაოდენობა და Telegram-ზე შეტყობინების გასაგზავნად საჭირო მონაცემები.

```
const float      FIRE_ON_THRESHOLD = 80.0f;
const float      FIRE_OFF_THRESHOLD = 55.0f;
const int        COLD_FRAMES_REQUIRED = 8;
const int        COOLING_FRAMES_REQUIRED = 10;
const float      COOLING_DELTA_THRESHOLD = -0.2f;
const unsigned long BEEP_INTERVAL_MS = 10000UL;
const unsigned long BEEP_DURATION_MS = 500UL;
const char*      TELEGRAM_BOT_TOKEN =
"7655520460:AAG239a4LqNq0acoINI3Pn9RyacG_6AhRUc";
String           TELEGRAM_CHAT_ID = "8154736889";
```

სისტემის სტატუსები (`state-ები`) და ნაკადებზე უსაფრთხო წვდომა

ეს ფრაგმენტი წარმოადგენს პროგრამის ნაკადების (`tasks`) უსაფრთხო მართვის მექანიზმს. სისტემას აქვს ოთხი სტატუსი (`state`): უმოქმედობა (`IDLE`), მონიტორინგი (`MONITORING`), ცეცხლის დაფიქსირება (`FIRE_ACTIVE`) და განგაში (`ALARM`). ცვლადი `currentState` ინახავს მიმდინარე სტატუსს, `getState()` კითხულობს მას, ხოლო `setState()` უსაფრთხოდ ცვლის ერთ მდგომარეობას მეორით. `stateMux` კი კრძალავს ორი ნაკადის ერთდროულ წვდომას ამ ცვლადზე და ამით გამოირიცხავს მონაცემთა კონფლიქტს - ეს მიიღწევა `portENTER_CRITICAL()` და `portEXIT_CRITICAL()` ფუნქციებით.

```
enum SystemState { STATE_IDLE, STATE_MONITORING, STATE_FIRE_ACTIVE,
STATE_ALARM };
```

```
static SystemState currentState = STATE_IDLE;
static portMUX_TYPE stateMux = portMUX_INITIALIZER_UNLOCKED;
```

```

inline SystemState getState() {
    SystemState s;
    portENTER_CRITICAL(&stateMux);
    s = currentState;
    portEXIT_CRITICAL(&stateMux);
    return s;
}

```

თერმული კამერის მუშაობის მექანიზმი

სენსორის ფუნქცია გაშვებულია პროცესორის მეორე ბირთვზე (**Core 1**). იგი კითხულობს 768-პიქსელიან (32×24) ტემპერატურულ მატრიცას, პოულობს მასში მაქსიმალურ მნიშვნელობას და გადასცემს მას სისტემის ძირითად მაკონტროლებელ ლოგიკას რიგის (queue) მეშვეობით.

```

inline void setState(SystemState s) {
    portENTER_CRITICAL(&stateMux);
    currentState = s;
    portEXIT_CRITICAL(&stateMux);
}

```

```

void sensorTask(void* parameter) {
    while (true) {
        if (getState() == STATE_IDLE) {
            vTaskDelay(50 / portTICK_PERIOD_MS);
            continue;
        }

        if (mlx.getFrame(sensorFrame) != 0) {
            Serial.println("[SENSOR] Frame read failed – retrying.");
            vTaskDelay(20 / portTICK_PERIOD_MS);
            continue;
        }

        float maxT = sensorFrame[0];
        for (int i = 1; i < 32 * 24; i++)
            if (sensorFrame[i] > maxT) maxT = sensorFrame[i];

        xQueueReset(tempQueue);
    }
}

```

```

xQueueSend(tempQueue, &maxT, 0);

vTaskDelay(10 / portTICK_PERIOD_MS);
}
}

```

ცეცხლის უეცარი ჩაქრობის ან ტემპერატურის თანდათანობითი ვარდნის დაფიქსირება

სისტემა ცეცხლის ჩაქრობას აფიქსირებს ორი პარალელური მეთოდით: (1) თუ ტემპერატურა ჩამოსცდება 55°C -ს; და (2) თუ ჭურჭელი ცხელ ტემპერატურას ინარჩუნებს, თუმცა შეინიშნება გაგრილების ტენდენცია: კადრებს შორის ტემპერატურის ვარდნა არის 0.2°C ან მეტი ზედიზედ რამდენიმე კადრის განმავლობაში.

```

case STATE_FIRE_ACTIVE: {
    if (maxTemp > peakTemp) peakTemp = maxTemp;

    float delta = maxTemp - previousTemp;

    if (delta <= COOLING_DELTA_THRESHOLD) {
        coolingFrames++;
    } else {
        coolingFrames = 0;
    }

    previousTemp = maxTemp;

    if (maxTemp < FIRE_OFF_THRESHOLD) {
        coldFrameCount++;
        if (coldFrameCount >= COLD_FRAMES_REQUIRED) {
            setState(STATE_ALARM);
            lastAlertTime = millis();
            startBuzzer();
            triggerAlert(maxTemp);
        }
    } else {
        if (coldFrameCount > 0) {
            coldFrameCount = 0;
        }
    }
}

```

```

    if (coolingFrames >= COOLING_FRAMES_REQUIRED) {
        setState(STATE_ALARM);
        lastAlertTime = millis();
        startBuzzer();
        triggerAlert(maxTemp);
    }

    break;
}

```

Telegram-შეტყობინების გაგზავნა

შეტყობინების გაგზავნის ფუნქცია გაშვებულია პროცესორის პირველ ბირთვზე (Core 0) და დამოუკიდებლად ამუშავებს Telegram API-ის მოთხოვნებს. ამის წყალობით, თერმული კამერიდან მიღებული მონაცემების დამუშავება არ ფერხდება შეტყობინების გაგზავნის დროს.

```

void telegramTask(void* parameter) {
    AlertData alert;
    while (true) {
        if (xQueueReceive(alertQueue, &alert, 0) == pdTRUE) {
            if (WiFi.status() == WL_CONNECTED) {
                WiFiClientSecure localClient;
                localClient.setInsecure();
                UniversalTelegramBot bot(TELEGRAM_BOT_TOKEN, localClient);

                String message = "🔥 *Gas Stove Alarm!*\\n";
                message += "The flame has gone out unexpectedly.\\n";
                message += "Last temperature reading: *" + String(alert.temp, 1) + " °C*\\n";
                message += "⚠️ Check the stove immediately — possible gas leak risk!";

                Serial.println("[Telegram] Sending alert...");
                bot.sendMessage(TELEGRAM_CHAT_ID, message, "Markdown");
                Serial.println("[Telegram] Alert sent.");
            } else {
                Serial.println("[Telegram] Skipped – no Wi-Fi.");
            }
        }
    }
}

```

```
    vTaskDelay(50 / portTICK_PERIOD_MS);  
  }  
}
```

სრული კოდის github-ის ბმული:

<https://github.com/MateMch-iliauni/Project-Codes/blob/main/finalproject.ino>